

API列表

API名称	描述
user/register	用户注册
user/unregister	用户注销
user/login	用户登录
device/bind	绑定设备
device/unbind	解绑设备
device/getDeviceList	获取设备列表
device/getLocationList	获取设备的位置列表
device/refreshLocation	刷新设备位置

API调用

请求地址

环境	HTTP请求地址
正式环境	device.vernal.ltd/tagapi

API 文档说明

API名称：register

描述

终端用户注册接口

请求信息

请求路径： /user/register

请求方法：POST

请求参数

名称	位置	类型	必填	描述

请求BODY

数据类型：

json

内容描述：

```
{
  "cid": "WE23dT7PmGUr02R+6Ch+GjMTmbw0qht6", //客户端id 必填
  "name": "11111111", //用户名 必填
  "pwd": "123456" //密码 必填
}
```

返回信息

返回参数类型

json

返回结果示例

```
{
  "code": "0000",
  "items": null,
  "message": "请求成功",
  "status": 200
}
```

API名称：login

描述

终端用户登录接口

请求信息

请求路径：/user/login

请求方法：POST

请求参数

名称	位置	类型	必填	描述

请求BODY

数据类型：

json

内容描述：

```
{
  "cid": "WE23dT7PmGUr02R+6Ch+GjMTmbw0qht6", //客户端id 必填
  "name": "111111", //用户名 必填
  "pwd": "123456" //密码 必填
}
```

返回信息

返回参数类型

json

返回结果示例

```
{
  "code": "0000",
  "items": {
    "id": 26, //用户id
    "name": "111111", //用户名
    "token": "S4xte+ypz1AcXa4xW1FLTbdijU1D8oeC" //用于请求加密用的token
  },
  "message": "请求成功",
  "status": 200
}
```

API名称：getDeviceList

描述

获取设备列表

请求信息

请求路径：/device/getDeviceList

请求方法：POST

请求参数

名称	位置	类型	必填	描述

请求BODY

数据类型：

json

内容描述：

```
{
  "userId": 26,      //用户ID      必填
  "data": "M/cnF490qMGdNzQNzIUfFI9iCe8FT0fhmKx+ZWVNB8k5M7oa8dhL4FZzbE2ABeCL6zRxvrt5dFXNUileTuMu/ZJAND
GQB2s2u2Z49dgZPE=" //加密后的用户数据  必填
}
```

加密前 data

```
{
  "pageNo": 1,      //页码
  "pageSize": 20,    //每一页数量
  "queryFilter": {
    "userId": 26     //用户id
  }
}
```

返回信息

返回参数类型

json

返回结果示例

```
{
  "status": 200,
  "code": "0000",
  "message": "操作成功",
  "items": "ctV99GK5rJHrdbLXN2laF3+BUaeYVA2c8/pMbEXsRLNTa80t3zSu7usoGKAnZKBevYSEQl6jb0ouRWyQIn0nTXufpQ
heNwTxADaUNpahu6hBc86mUsiZ4ccmctw/1rn6G1PzKJW5iWDatRaNE4+6wqVKVQALGMlngfAqUS9FEY5SC0xR/uj/pSobaQfsw981rz
0zP8FM8LPPXC4658U5EfEYQogYHAvmfRQj3shh+NupwdYDjNg01fHBgEU8ykJI9vNpjrYY0vj28MHOKBLs/hDeOS634BCx5FWxz1tx1x
HkNnZh7C3BIXV0S0iU+XnYRxs3yBoUh11BAX9kVV1iINkX2F+tG078GYybGj6CC9p0AJg7aBaKJOUdQDlQ5d7y2RD40aNvKwFwtE+0q
e6G1edgSk0WxKB9vDBzigS7P4Q3jkuT+AQseRVsc5bcZcR5DZ2YX+wtwRSaOnwGA18YVNHxaneBQFmdQQF/ZFVZYiDZF9hfrRt000ffs
JMST4z7Hu40/mySOAS020oTMCjjstkQ+NGjbysBcLRPtKnuhuN5Es/Qz3PXfbwwc4oEuz+EN45LrfgELHkVbHOW3GXEEQ2dmF/sLcE5M
h33L1iUHKHb6EcEF10fHUEBF2RVWMIg2RfYX60bTupVRCH0q623aKRokOnKjGC7t01tw2jm53LZEPjRo28rAXC0T7Sp7obgU8wpEGowD
P28MHOKBLs/hDeOS634BCx5FWxz1tx1xHkNnZh7C3BCDxYgpIY6EZqNfGSHC/oVt1BAX9kVV1iINkX2F+tG07M8f/iz6AX29ge50MGy
VIkKIvrhva6rzby2RD40aNvKwFwtE+0qe6G1IpWS0Amego9vDBzigS7P4Q3jkuT+AQseRVsc5bcZcR/WyReaJ67kBB1CDcX5FYA5q1xK
20JCLYdQQF/ZFVZYiDZF9hfrRt07yoG7ne49wKv0mjmBqfT46VemY1WwjYtMtkQ+NGjbys8znFbezI+i81X1ZvP337XgL23c/S1t413U
VvuJTKQSpjntj9x7/ozejXGwoMNJQbwfMcc5LP0DUeGcn6q+ZqDA=="
}
```

items 解密后数据

```
{
  "current": 1,    //当前页码
  "pages": 1,     //总共页数
  "records": [{
    "deviceNum": "1712591093199087",    //设备号
    "location": {      //设备最新一条数据
      "latitude": "29.3646496",    //纬度
      "longitude": "105.9406226",  //经度
      "timestamp": "1742873587000", //时间
      "battery": 100    //电量
    },
    "mac": "F1:1F:D9:16:D1:C2",    //MAC 地址
    "nickName": "bike",           //设备名
    "userId": 26                  //用户id
  }, {
    "deviceNum": "1716953170364070",
    "location": {},
    "mac": "C1:F1:3C:13:BE:24",
    "nickName": "方板子4号",
    "userId": 26
  }, {
    "deviceNum": "1716953170801801",
    "location": {},
    "mac": "E0:14:E2:29:44:E5",
    "nickName": "方板子2号",
    "userId": 26
  }, {
    "deviceNum": "1716953172761163",
    "location": {},
    "mac": "E4:A9:31:08:A6:B7",
    "nickName": "方板子3号",
    "userId": 26
  }, {
    "deviceNum": "1716953173985356",
    "location": {},
    "mac": "FA:E7:03:8B:67:A1",
    "nickName": "方板子1号",
    "userId": 26
  }, {
    "deviceNum": "1726482692700617",
    "location": {},
    "mac": "F0:D6:01:D4:A8:F4",
    "nickName": "MOTO",
    "userId": 26
  }
  ],
  "searchCount": true,
  "size": 6,
  "total": 6
}
```

API名称: getLocationList

描述

获取定位列表

请求信息

请求路径: /device/getLocationList

请求方法: POST

请求参数

名称	位置	类型	必填	描述

请求BODY

数据类型:

json

内容描述：

```
{
  "userId": 26, //用户ID      必填
  "data": "M/cnF490qMGdNzQNzIUfFI9iCe8FT0fhmKx+ZWwVNB8k5M7oa8dhL4FZzbE2ABeCL6zRxvrt5dFV6LMUzMhyXtFfL2K/TfzTvYSEQl6jb0OuRwyQInOnTaK990inqhLV9vDBzigS7P7THkIq9Y9vSL3P+JUCvCK/" //加密后的用户数据      必填
}
```

加密前 data

```
{
  "pageNo": 1,
  "pageSize": 20,
  "queryFilter": {
    "deviceNum": "1712591093199087",
    "userId": 26,
    "beginTime": 1752271117000, //开始时间
    "endTime": 1752299917000    //结束时间
  }
}
```

返回信息

返回参数类型

json

返回结果示例

```
{
  "status": 200,
  "code": "0000",
  "message": "操作成功",
  "items": "ctV99GK5rJHrdbLXN2laF3+BUaeYVA2c8/pMbEXsRLPk81Ulmr76yfSBik7KIUNBpzQD0Li03VjYfyZnXp8k3/UEstvAaqkleXMonfeQc3aVb6xHzmBoJXBAGUU1Wtauo96Ah02DX8J6Wla/ynQG1WuW0zEQ327vsx57F0xsR+hDKF5eY64dM2WkLhmSWKvmq+oLtcH6DkVF/9rj/Ke+uTu/fpoX7n9V2gCPmSQcPtbgIgfPQdaKBVWI3gan0Qvmq+oLtcH6DvxM2ltwLRlPe0tekOVorNquMzI507VJE7bQ20nXL4qrLndSIygwXU7PCCI1uIeUsjd3cUz/9kw/beeESuAQP7wS+6YvDoiWNWFPEM56Zex1rrCvjGp2sGIgX16D9fyYYNlrsWMrvTMNh/JmdenyTf9QSy28BqqSXpsPC2LzmB4ZVvrEfOYGglcECQZRTVa1oJDBp+w3oTFgnpaVr/KdAbVa5bTMRDfbuKkKw/QwuGf05d3PJcba16sL+QV2dPZlAmgQ4SzyYnfkco0tHRkxcc4dRgaXmg3WW9y0nIkoeqRJX1svrsqkLa0gQTWJ0xi1ud0az0vw0NK2Q=="
}
```

items 解密后数据

```
{
  "current": 1,
  "pages": 1,
  "records": [{
    "accuracy": "49", //精度
    "datePublished": 1742873999000, //数据上报时间
    "latitude": "29.3646496", //纬度
    "longitude": "105.9406226", //经度
    "timestamp": 1742873587000, //时间
    "battery": 100 //电量
  }, {
    "accuracy": "48",
    "datePublished": 1742873585000,
    "latitude": "29.364612",
    "longitude": "105.9401178",
    "timestamp": 1742873584000,
    "battery": 100 //电量
  }, {
    "accuracy": "131",
    "datePublished": 1742871672000,
    "latitude": "29.3647014",
    "longitude": "105.940289",
    "timestamp": 1742871119000,
    "battery": 100 //电量
  }],
  "searchCount": true,
  "size": 3,
  "total": 3
}
```

API名称：refreshLocation

描述

请求刷新设备位置

请求信息

请求路径：/device/refreshLocation

请求方法：POST

请求参数

名称	位置	类型	必填	描述

请求BODY

数据类型：

json

内容描述：

```
{
  "userId": 3, //用户 ID      必填
  "data": "BIH80dqjWdLPxxnHm1mGawDtsryFV5XTUYQ3pqWodckxWIsrwFZW8V4JyPZRHHpxbaIX4jm1tATiGBdLiREvZB4/THcBsfdTQVsBU9CuVWp9dWJCUiw/9Ud8U9vJx89ZclfEvSI9+LM=" //加密后的用户数据      必填
}
```

加密前 data

```
{
  "cid": "Gk7F6n92incFPoKRJCdpxK3YWgKiWw41",
  "deviceNum": "1716991605662459",
  "userId": 3
}
```

返回信息

返回参数类型

json

返回结果示例

```
{
  "status": 200,
  "code": "0000",
  "message": "操作成功",
  "items": null
}
```

API名称：bind

描述

绑定设备

请求信息

请求路径：/device/bind

请求方法：POST

请求参数

名称	位置	类型	必填	描述

请求BODY

数据类型：

json

内容描述：

```
{
  "userId": 26, //用户ID 必填
  "data": "dmfICKsRMWRDLcW6AKV710jk8JIazaITIj6LjPrQF5Wazx1bM3VLW03YnsLjPdTHR5J/LYX874cCTm7+I1R91A==" /
  /加密后的用户数据 必填
}
```

data加密前数据(deviceNum 与sn二选一)

```
{
  "cid": "WE23dT7PmGUr02R+6Ch+GjMTmbw0qht6", //客户端id 必填
  "deviceNum": "1726482692700617", //设备号(扫描二维码)
  "sn": "dfffd47c82f64", //sn(通过协议获取)
  "userId": 26
}
```

返回信息

返回参数类型

json

返回结果示例

```
{
  "code": "0000",
  "items": {
    "mac": "FF:FF:FF:FF:FF:FF"
  },
  "message": "请求成功",
  "status": 200
}
```

API名称：unbind

描述

解绑设备

请求信息

请求路径：/device/unbind

请求方法：POST

请求参数

名称	位置	类型	必填	描述

请求BODY

数据类型：

json

内容描述：

```
{
  "userId": 26, //用户ID 必填
  "data": "dmfICKsRMWRDLcW6AKV71G3bnB8mursMQZQg3F+RWAMiAsp8I330Gvbwwc4oEuz+AKbFa3jayFw=" //加密后的用户数据 必填
}
```

加密前 data

```
{
  "cid": "WE23dT7PmGUrO2R+6Ch+GjMTmbwOqht6", //客户端id 必填
  "deviceNum": "1726482692700617",
  "userId": 26
}
```

返回信息

返回参数类型

json

返回结果示例

```
{
  "code": "0000",
  "items": null,
  "message": "请求成功",
  "status": 200
}
```

API名称：unregister

描述

终端用户注销接口

请求信息

请求路径：/user/unregister

请求方法：POST

请求参数

名称	位置	类型	必填	描述

请求BODY

数据类型：

json

内容描述：

```
{
  "userId": "1", //用户ID 必填
  "data": "W9qwRTwEc4ZHkn8thfzvhwJObv4iVH3U" //加密后的用户数据 必填
}
```

加密前 data

```
{
  "cid": "WE23dT7PmGUrO2R+6Ch+GjMTmbwOqht6", //客户端id 必填
  "userId": 26
}
```

返回信息

返回参数类型

json

返回结果示例

```
{
  "code": "0000",
  "items": null,
  "message": "请求成功",
  "status": 200
}
```


错误码

错误码	错误信息	描述
8001	邮件地址不正确	
8002	验证码已过期	
8003	验证码错误	
8004	验证码已发送	
8005	请先获取验证码	
8006	用户已存在	
8007	用户注册失败	
8008	用户注册成功	
8009	用户不存在	
8010	密码错误	
8011	解密错误,请重新获取token	
8012	设备绑定失败	
8013	设备不存在	
8014	绑定失败,设备已绑定	
8015	用户认证失败	
8016	请输入设备号	
8017	设备二维码无效	
8018	未知错误	
8019	客户端未授权	
8020	设备与客户端不匹配	
8026	设备未绑定	

加解密

java

```

public class DESUtils {

    /**
     * Triple DES 加密
     *
     * @param plainText 需要加密的明文
     * @param key 加密密钥（必须是24字节）
     * @return 返回 Base64 编码的加密结果
     */
    public static String encrypt(String plainText, String key) throws Exception {
        // 将密钥转换为字节数组
        byte[] keyBytes = key.getBytes(StandardCharsets.UTF_8);

        // 创建 DESedeKeySpec
        DESedeKeySpec spec = new DESedeKeySpec(keyBytes);
        SecretKeyFactory keyFactory = SecretKeyFactory.getInstance("DESede");
        SecretKey secretKey = keyFactory.generateSecret(spec);

        // 初始化 Cipher 为加密模式
        Cipher cipher = Cipher.getInstance("DESede/ECB/PKCS5Padding");
        cipher.init(Cipher.ENCRYPT_MODE, secretKey);

        // 加密数据
        byte[] encryptedBytes = cipher.doFinal(plainText.getBytes(StandardCharsets.UTF_8));

        // 返回 Base64 编码的加密结果
        return Base64.getEncoder().encodeToString(encryptedBytes);
    }

    /**
     * Triple DES 解密
     *
     * @param encryptedText Base64 编码的加密字符串
     * @param key 解密密钥（必须是24字节）
     * @return 返回解密后的明文
     */
    public static String decrypt(String encryptedText, String key) throws Exception {
        // 将密钥转换为字节数组
        byte[] keyBytes = key.getBytes(StandardCharsets.UTF_8);

        // 创建 DESedeKeySpec
        DESedeKeySpec spec = new DESedeKeySpec(keyBytes);
        SecretKeyFactory keyFactory = SecretKeyFactory.getInstance("DESede");
        SecretKey secretKey = keyFactory.generateSecret(spec);

        // 初始化 Cipher 为解密模式
        Cipher cipher = Cipher.getInstance("DESede/ECB/PKCS5Padding");
        cipher.init(Cipher.DECRYPT_MODE, secretKey);

        // 解码 Base64 并解密数据
        byte[] decryptedBytes = cipher.doFinal(Base64.getDecoder().decode(encryptedText));

        // 返回解密后的明文
        return new String(decryptedBytes, StandardCharsets.UTF_8);
    }
}

```

js

```

const crypto = require('crypto');

/**
 * Triple DES 加密
 * @param {string} plainText 需要加密的明文
 * @param {string} key 加密密钥（必须是24字节）
 * @returns {string} 返回 Base64 编码的加密结果
 */
function encrypt(plainText, key) {
  // 检查密钥长度（3DES需要24字节）
  if (Buffer.from(key, 'utf8').length !== 24) {
    throw new Error('Key must be 24 bytes long for Triple DES');
  }

  // 创建cipher对象
  const cipher = crypto.createCipheriv('des-ede3', Buffer.from(key, 'utf8'), null);

  // 加密数据
  let encrypted = cipher.update(plainText, 'utf8', 'base64');
  encrypted += cipher.final('base64');

  return encrypted;
}

/**
 * Triple DES 解密
 * @param {string} encryptedText Base64 编码的加密字符串
 * @param {string} key 解密密钥（必须是24字节）
 * @returns {string} 返回解密后的明文
 */
function decrypt(encryptedText, key) {
  // 检查密钥长度（3DES需要24字节）
  if (Buffer.from(key, 'utf8').length !== 24) {
    throw new Error('Key must be 24 bytes long for Triple DES');
  }

  // 创建decipher对象
  const decipher = crypto.createDecipheriv('des-ede3', Buffer.from(key, 'utf8'), null);

  // 解密数据
  let decrypted = decipher.update(encryptedText, 'base64', 'utf8');
  decrypted += decipher.final('utf8');

  return decrypted;
}

module.exports = { encrypt, decrypt };

```

CryptoJS

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/crypto-js/4.1.1/crypto-js.min.js"></script>
<script>
  /**
   * Triple DES 加密
   */
  function encrypt(plainText, key) {
    const keyHex = CryptoJS.enc.Utf8.parse(key);
    const encrypted = CryptoJS.TripleDES.encrypt(plainText, keyHex, {
      mode: CryptoJS.mode.ECB,
      padding: CryptoJS.pad.Pkcs7
    });
    return encrypted.toString();
  }

  /**
   * Triple DES 解密
   */
  function decrypt(encryptedText, key) {
    const keyHex = CryptoJS.enc.Utf8.parse(key);
    const decrypted = CryptoJS.TripleDES.decrypt(encryptedText, keyHex, {
      mode: CryptoJS.mode.ECB,
      padding: CryptoJS.pad.Pkcs7
    });
    return decrypted.toString(CryptoJS.enc.Utf8);
  }
</script>
```